



ELEO
FARM & FOODS

Campus Egg Commerce Platform

24-Hour Progressive Web App MVP | Product & Engineering Proposal

PREPARED FOR

ELEO Farm & Foods - Tech Team

VERSION

MVP v1.0 | September 2026

SECTION 1

Overview & Objective

Build a mobile-first, installable Progressive Web App that lets students order eggs digitally, and lets the ELEO administrator manage products, payments and fulfilment from one dashboard.

Tagline on landing screen: "Fresh Eggs. Simple Ordering."
Success test: a student can go from opening the PWA to a completed, tracked order, and the admin can process it end to end, with no manual workarounds.

Success at a glance

Area	Target
Users	Students (customers) and one business administrator
Device	Phone first; installable to home screen as a PWA
Payment	Manual bank transfer with admin confirmation (no gateway)
Timeline	24-hour build; complete customer-to-admin flow is the priority
Data rule	No hard-coded prices in the frontend; everything reads from the database

Scope

Must work	Nice to have	Do not build
Product catalogue	Goal selection	AI nutrition
Cart	Basic nutrition page	GPS
Checkout	Reorder button	Native mobile apps
Order creation	Revenue dashboard	Multi-vendor marketplace
Bank transfer instructions		Automated payment gateway
Payment confirmation		Complex analytics
Admin dashboard		Route optimisation
Order status tracking		Fitness tracking
Customer order history		Subscriptions
PWA installation		

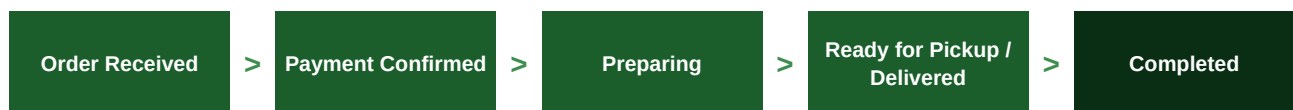
SECTION 2

Customer Experience

Screens and behaviour for the student-facing app.

Screen	What it shows / does
Landing	Headline "Fresh Eggs. Simple Ordering." with three buttons: Order Eggs, My Orders, My Profile.
Product catalogue	Live products from the database: name, description, quantity, price, availability. Unavailable items are visibly disabled.
Product selection	Customer picks a product and quantity, then taps Add to Cart.
Cart	Product, quantity, unit price, subtotal, delivery fee (if applicable) and total. Buttons: Continue Shopping, Checkout.
Checkout	Collects name, phone, hostel, room/block, fulfilment method (Pickup or Delivery) and an optional note. Submitting generates an order number, e.g. EGG-000124 .
Payment	Shows bank name, account name, account number, amount and order number. Button "I've Made Payment" sets <code>payment_status = pending_confirmation</code> .
Order tracking	Customer sees the current order status at all times.
My Profile	Name, phone, hostel, room, goal. Lists current orders, previous orders, order details and Reorder.
Nutrition page	Goal-based short message with an Order Eggs button (see below).

Order lifecycle



The admin changes status manually. Orders can also be Cancelled by the admin.

Customer goals & basic nutrition page

Goals: Build muscle, Gain weight, Lose weight, Maintain weight, Eat healthier. Each goal shows one short, non-medical line, for example: *"Eggs are a source of protein and can form part of a balanced diet."* followed by an Order Eggs button. No calorie calculators, AI, medical advice or workout tracking.

SECTION 3

Admin Dashboard

Secure, login-protected area for managing the business day to day.

Dashboard metrics

Metric	Definition
Today's orders	Count of orders created today
Pending payments	Orders with payment_status = pending_confirmation
Orders preparing	order_status = preparing
Orders ready	order_status = ready
Completed orders	order_status = completed
Revenue	Sum of totals for confirmed / completed orders

Orders view

Each order row shows: order number, customer, phone, products, quantity, total, hostel, room, fulfilment method, payment status, order status, date/time.

Admin action	Result
Confirm Payment	payment_status = confirmed; order_status = payment_confirmed
Mark Preparing	order_status = preparing
Mark Ready	order_status = ready (pickup)
Mark Delivered	order_status = delivered / completed for delivery orders
Cancel	order_status = cancelled

Product management

- Add and edit products; change price and quantity.
- Toggle available / unavailable.
- Changes appear on the customer catalogue automatically (single source of truth in the database).

SECTION 4

Database Design

Four core tables. Suggested types are shown for Supabase / PostgreSQL.

users

Column	Type	Notes
id	uuid (PK)	Linked to Supabase Auth
name	text	
phone	text	
hostel	text	
room	text	
goal	text / enum	muscle, gain, lose, maintain, healthier
created_at	timestampz	default now()

products

Column	Type	Notes
id	uuid (PK)	
name	text	
description	text	
quantity	integer	Pack size or stock (confirm with team)
price	numeric(10,2)	Never hard-coded in UI
available	boolean	Controls catalogue visibility
created_at	timestampz	

orders

Column	Type	Notes
id	uuid (PK)	
order_number	text unique	EGG-000124 from a DB sequence
user_id	uuid (FK)	References users.id
total	numeric(10,2)	
payment_status	enum	unpaid, pending_confirmation, confirmed
order_status	enum	received, payment_confirmed, preparing, ready, delivered, completed, cancelled
fulfillment_method	enum	pickup, delivery
delivery_location	text	Hostel + room/block snapshot
created_at	timestampz	

order_items

Column	Type	Notes
id	uuid (PK)	
order_id	uuid (FK)	References orders.id
product_id	uuid (FK)	References products.id
quantity	integer	
unit_price	numeric(10,2)	Price snapshot at order time

SECTION 5

Recommended Stack & Architecture

Use whatever the team can ship fastest. The recommended stack is below.

Layer	Choice	Why
Frontend	Next.js	Routing, server rendering and PWA support out of the box
Styling	Tailwind CSS	Fast, consistent mobile-first UI
Backend / DB	Supabase (PostgreSQL)	Auth, database and row-level security in one service
Hosting	Vercel	One-click deploys and preview URLs
App type	PWA	Web app manifest + service worker, installable, no app store

PWA requirements

- Web app manifest with ELEO name, theme colour (deep green), icons at 192 and 512 px.
- Service worker for app shell caching; served over HTTPS.
- Mobile-first layouts and large tap targets; verify Add to Home Screen on Android and iOS.

Recommended security & data rules

- Admin routes protected by an admin role; never expose admin actions to customers.
- Row-level security so students only read their own orders.
- Calculate order totals on the server from database prices, not from client input.
- Store `unit_price` on each order item so later price edits do not alter past orders.

SECTION 6

24-Hour Build Plan

Suggested sequence prioritising the critical demo path.

Hours	Focus	Deliverable
0 - 3	Setup	Repo, Supabase project, schema, seed products, Vercel deploy, PWA manifest
3 - 8	Storefront	Landing, catalogue, product selection, cart, checkout form
8 - 12	Orders & payment	Order creation, order number, bank details screen, "I've Made Payment"
12 - 17	Admin	Login, dashboard, orders list, payment confirmation, status actions, product CRUD
17 - 20	Tracking & history	Customer order status, order history, order details
20 - 22	Nice to have	Goals, nutrition page, reorder, revenue tile
22 - 24	QA & demo	Full end-to-end run on real phones, install test, bug fixes

Critical demo (acceptance test)

Student	Admin
1. Open PWA 2. Browse eggs 3. Select quantity, add to cart 4. Checkout 5. See bank details 6. Tap I've Made Payment; receive order number 7. Track order 8. See Payment Confirmed, Preparing, Ready for Pickup, Completed 9. View order in history	1. Login 2. See new order 3. Confirm payment 4. Mark preparing 5. Update status to ready 6. Complete order

Definition of done: the full flow above works from start to finish on a real phone. Everything else comes after it.

Open questions for the team

- **Delivery fee:** the cart shows a delivery fee but the orders table has no column for it. Add `delivery_fee` (or a settings table).
- **Customer note:** checkout collects an additional note; add note to orders.
- **Products.quantity:** confirm whether this means pack size (e.g. crate of 30) or stock on hand.
- **Bank details:** store in a settings table so ELEO can update them without a deploy.
- **Guest checkout:** decide whether students must sign up before ordering or can order with just phone number.